INT1405 - DISTRIBUTED SYSTEMS

LECTURE 2

Communication and Naming

Semester 1, 2026-2027

Lê Hà Thanh
lehathanh@ptithcm.edu.vn

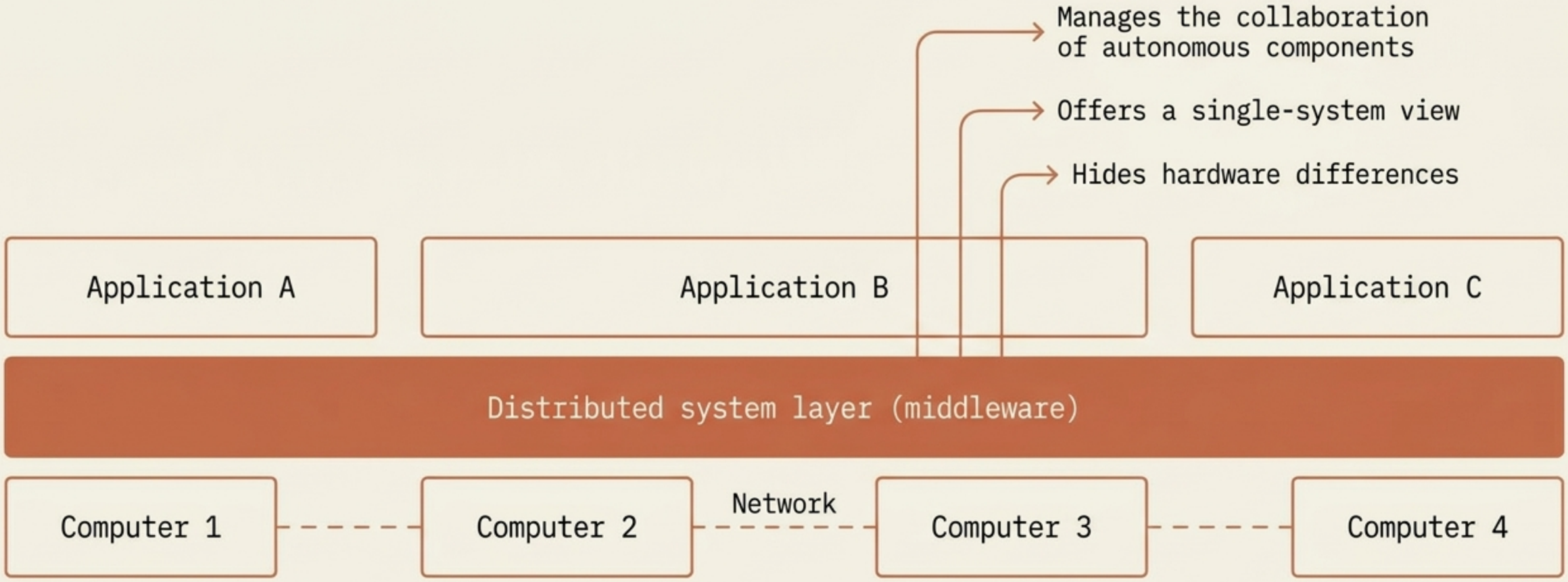
Tanenbaum & van Steen (2nd Edition),
Chapter 4 and 5

PTIT
POSTS AND TELECOMMUNICATIONS
INSTITUTE OF TECHNOLOGY

Content

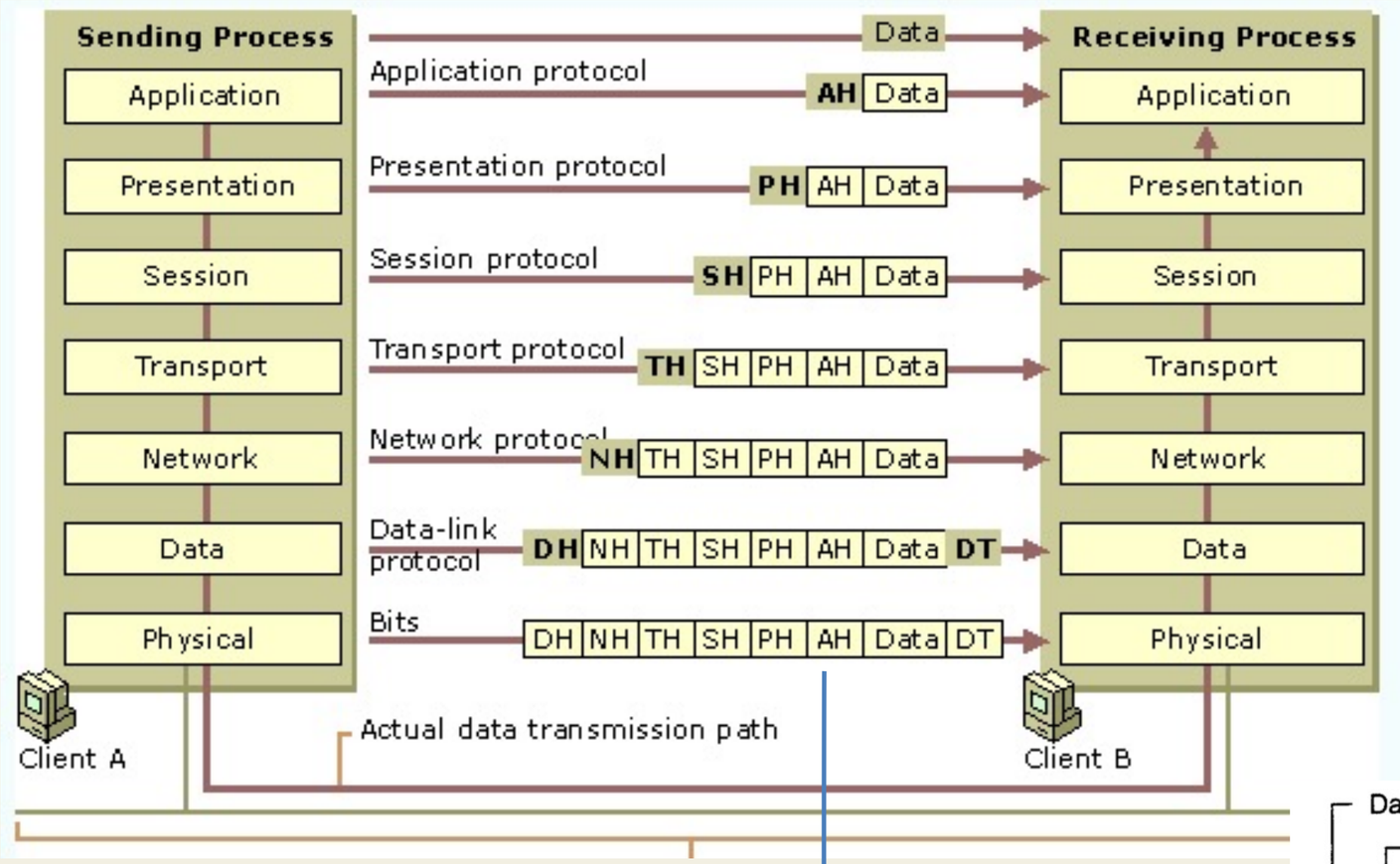
- Communication Protocols
- Remote Procedure Call (RPC)
- Messaging
- Naming Systems

Layer Protocol Stack & Middleware

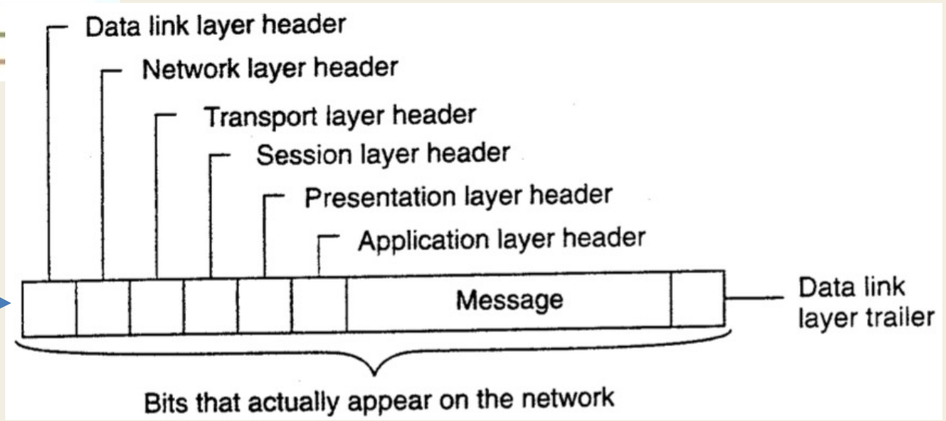


How do independent nodes talk, and how do they know who to talk to?

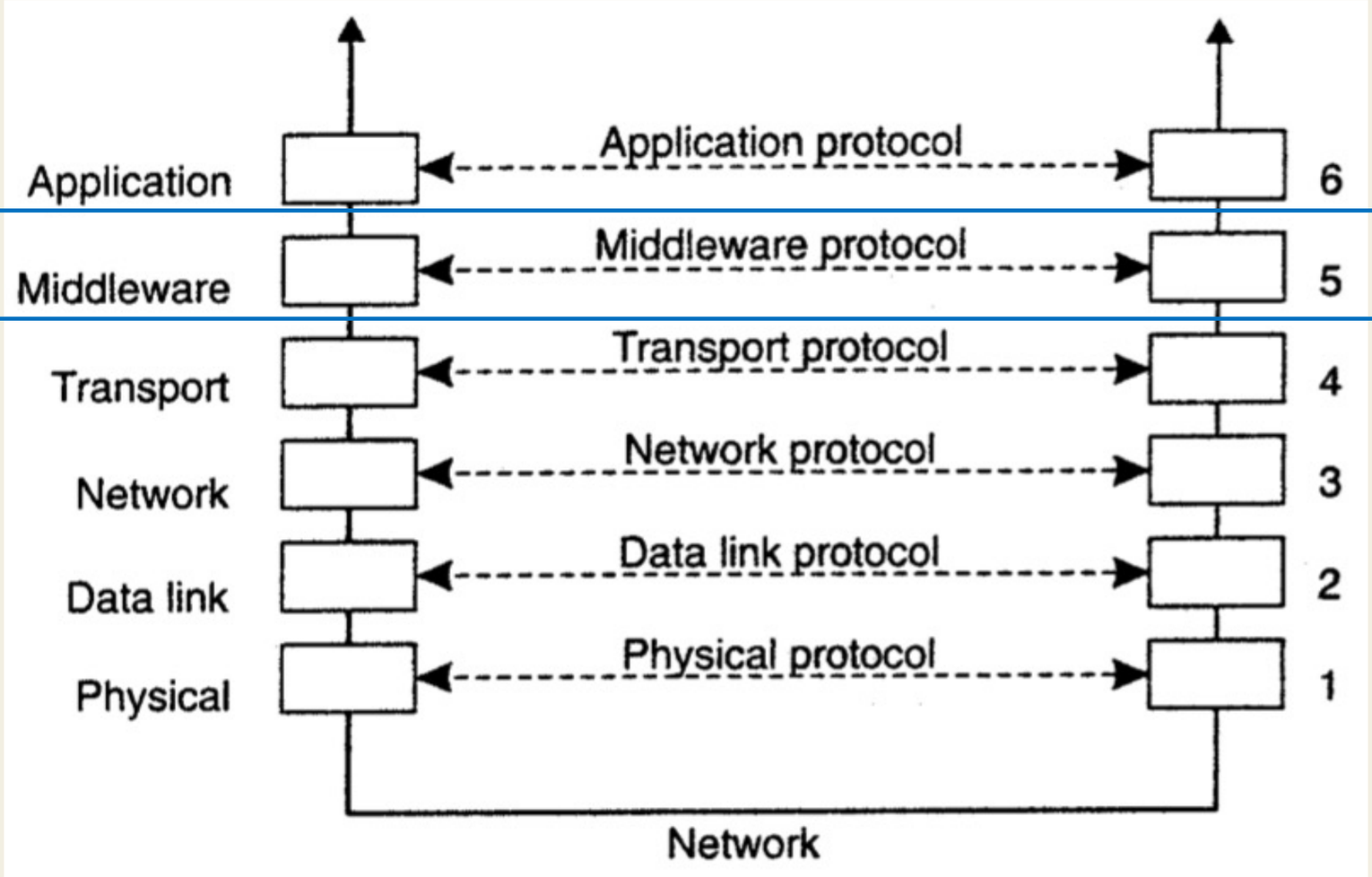
Network: OSI model



Description	Example protocols	Message Name
Network process to application.	HTTP, FTP, SSH, Telnet, DNS, etc.	Data
Recognizing data.	HTML, MPEG, JPEG, IMAP, TLS, etc.	
Session establishment and termination.	Sockets, Winsock, APIs, etc.	
End-to-end connectivity and reliability.	TCP, UDP	Segment (TCP) Datagram (UDP)
Logical addressing (IP) and path determination.	IPv4, IPv6, ICMP, IPsec, IGMP, OSPF, etc.	Packet
Physical addressing (MAC).	802.3 Ethernet, 802.11 Wireless, PPP, HDLC, etc.	Frame
Media, signals and binary transmision.	100Base-FX,1000Base-LX, RS-232, DSL, etc.	Bits

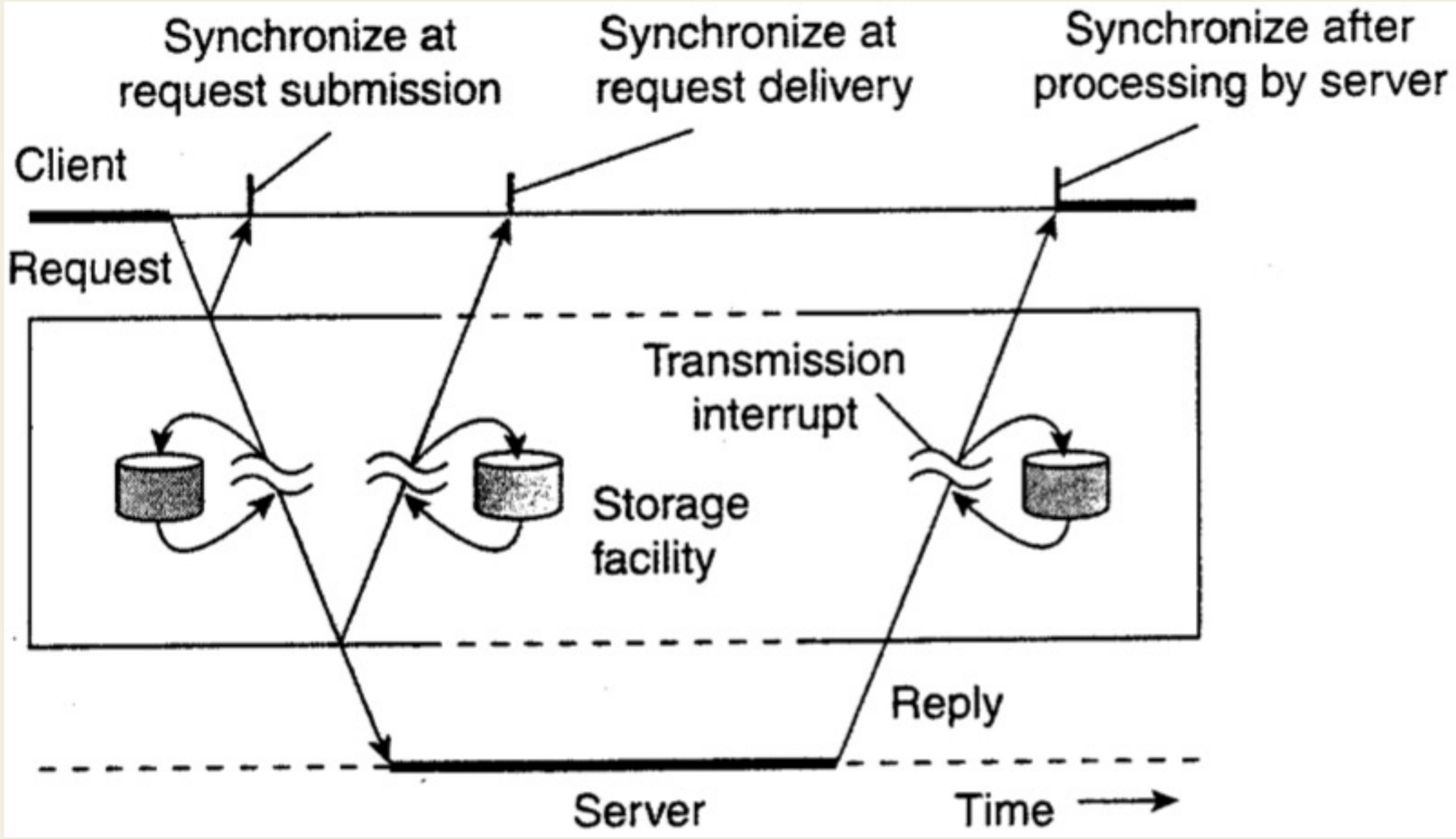


Communication middleware

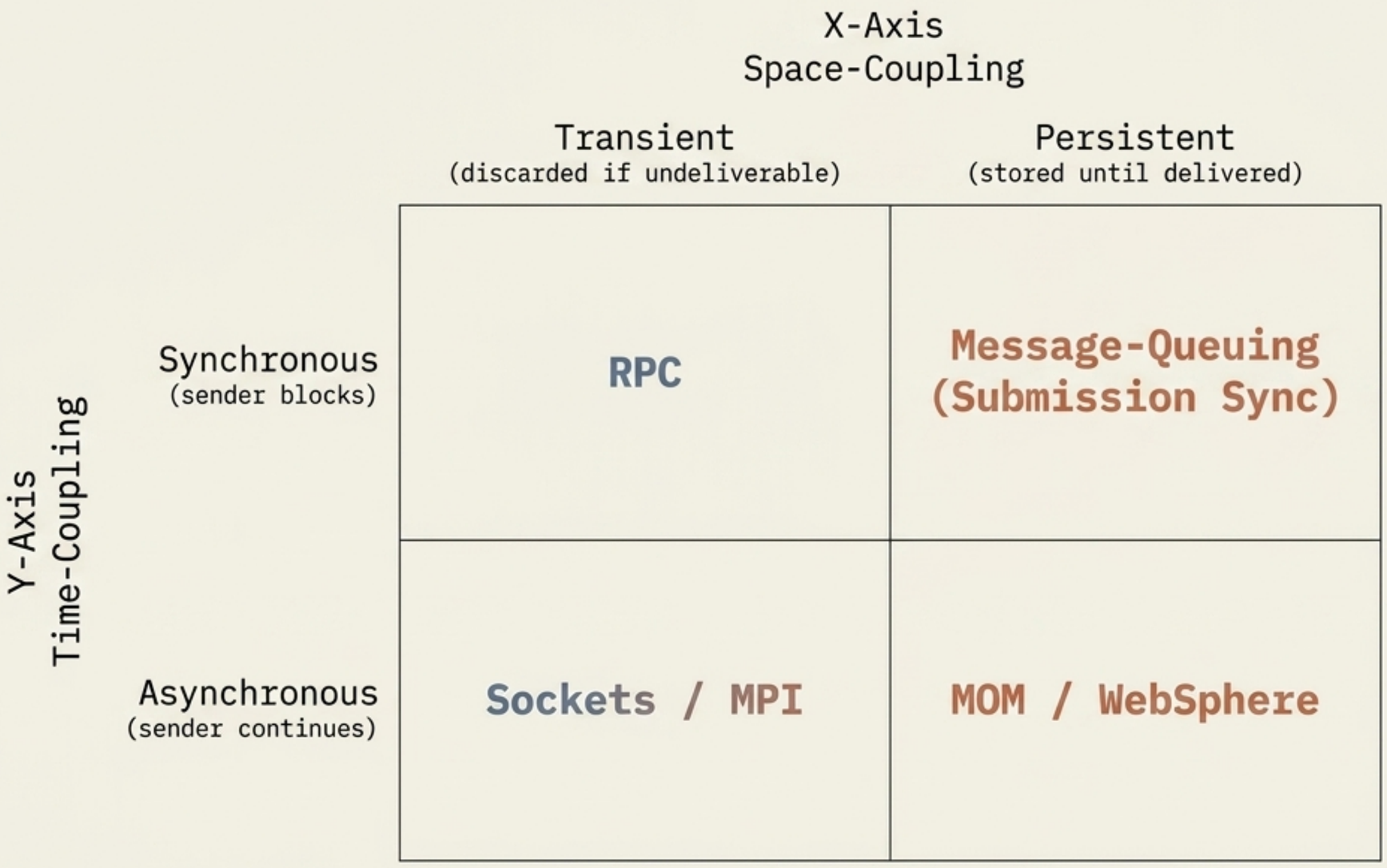


Adapted Reference Model for Communication Middleware

Communication middleware: Persistent vs. Transient



Middleware as an intermediate (distributed) service in application-level communication


X-Axis
Space-Coupling
Transient
(discarded if undeliverable)
Persistent
(stored until delivered)
Y-Axis
Time-Coupling
Synchronous
(sender blocks)
Asynchronous
(sender continues)
RPC
Message-Queuing
(Submission Sync)
Sockets / MPI
MOM / WebSphere

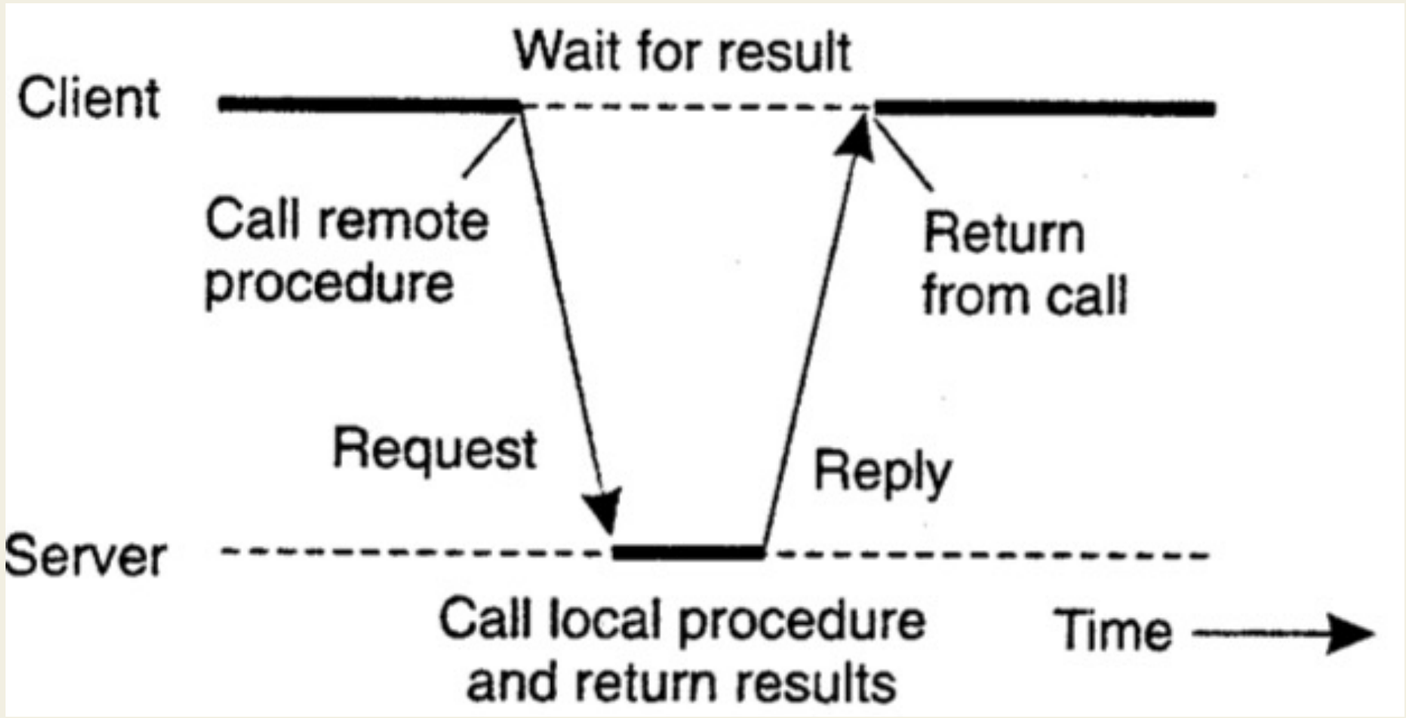
Content

- Communication Protocols
- Remote Procedure Call (RPC)
- Messaging
- Naming Systems

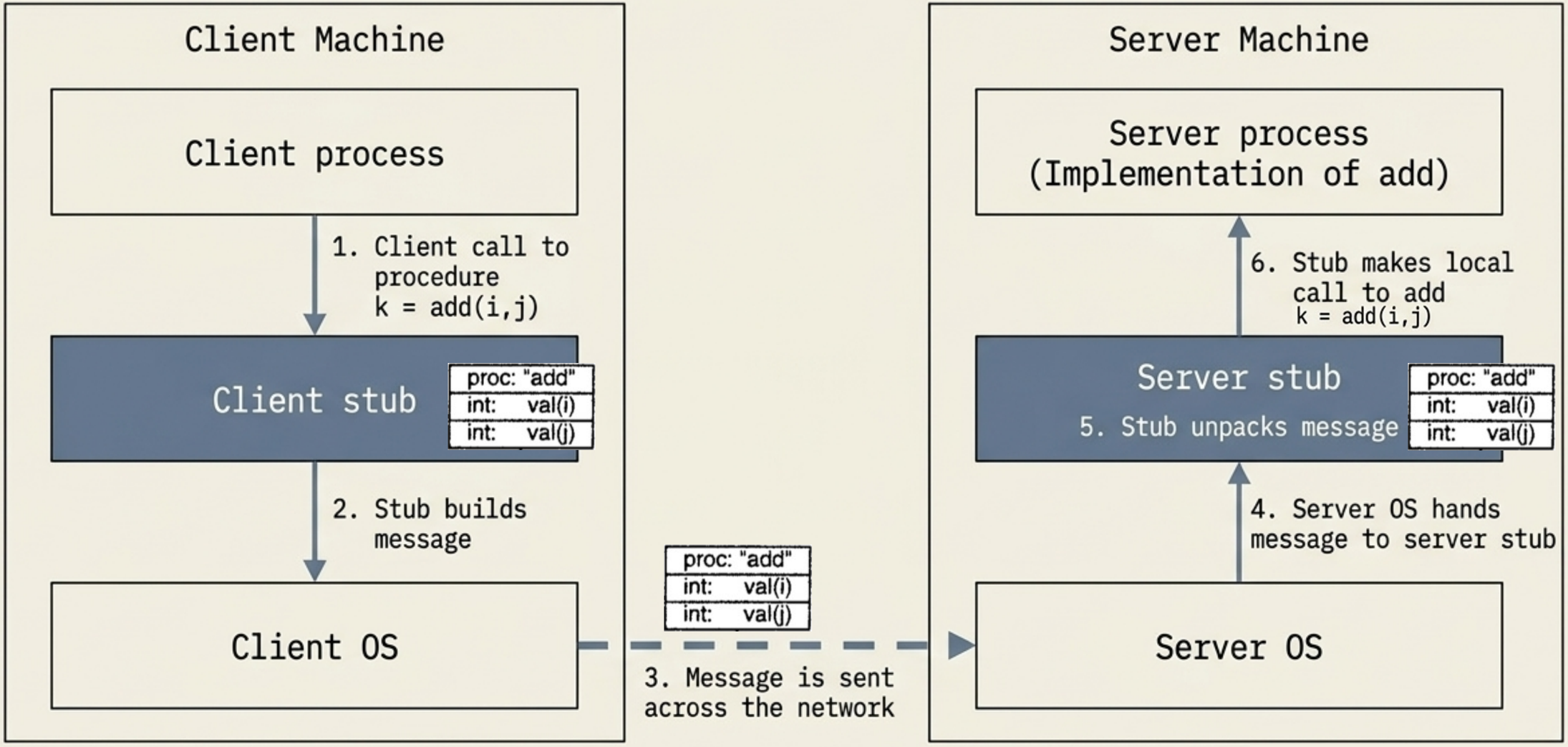
Remote Procedure Call (RPC)

- Introduced by Birrell and Nelson (1984) to make remote calls behave identically to local calls.
- Hides message-passing mechanisms under procedure-call semantics
- Client & Server Stubs:
  - Client stub marshals procedure parameters into a network request;
  - Server stub unmarshals parameters and invokes the local target procedure

RPC between client and server



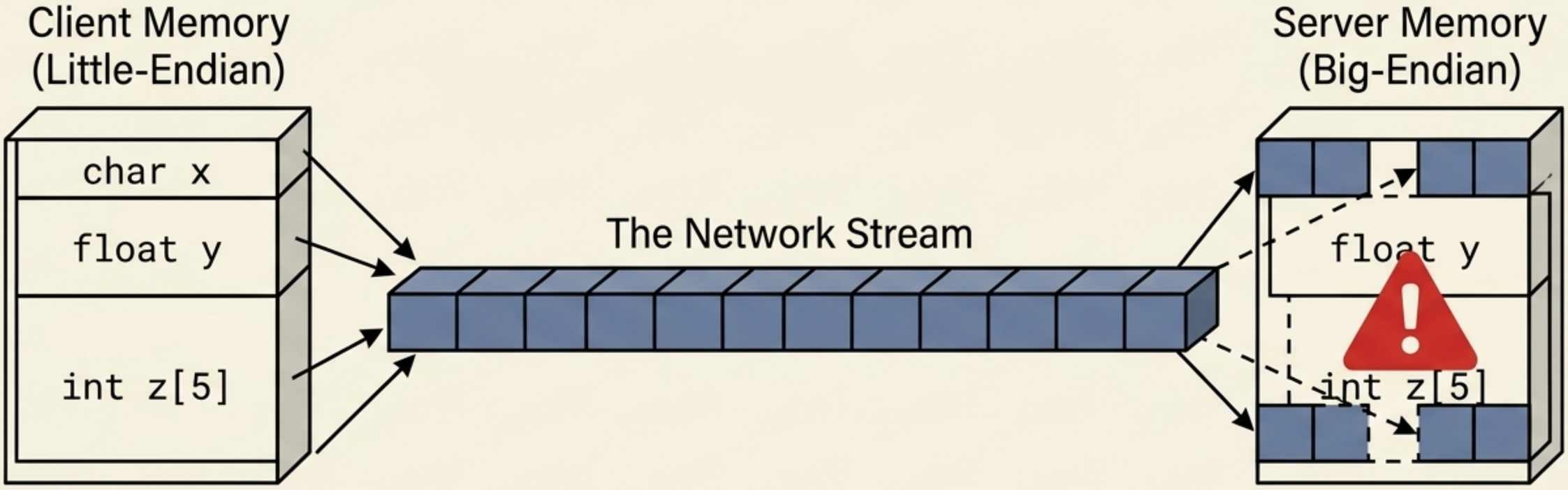
Executing a Remote Computation via RPC: Client & Server Stubs



RPC Parameter Passing & Marshaling

- **Heterogeneity Challenges**: Differing byte ordering (Big-Endian vs. Little-Endian) and character encodings (ASCII vs. EBCDIC).
- **Marshaling**: Packing machine-specific data structures into a flat, machine-independent wire format
- **Pointers & References**: Passed via copy-restore semantics or replaced by globally unique remote references
- Interface Definition Language (IDL): Defines procedure signatures compiled into matching client/server stubs

The Parameter Marshaling Chasm



Both sides must strictly adhere to the generated stub protocol; otherwise, the illusion breaks.

Asynchronous & One-Way RPCs

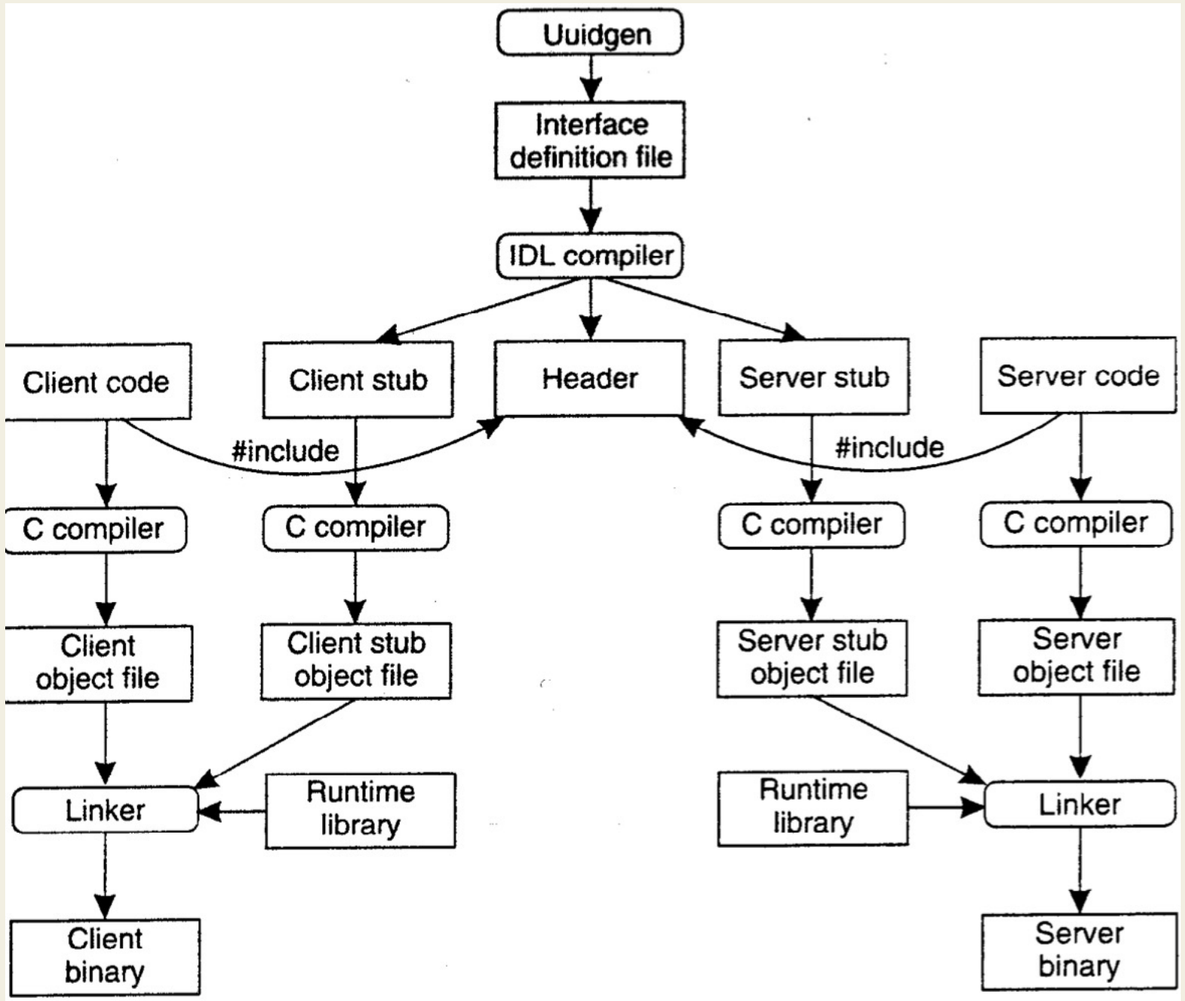
- Standard RPC blocks the client while waiting for a server response.
- **Asynchronous RPC**: Server acknowledges message receipt immediately; client resumes execution without waiting for the final computation result.
- **One-Way RPC / Deferred Synchrony**: Client sends a request and continues; results are retrieved later via a callback or polling handle

Real-World Case Study: DCE RPC & Endpoint Binding

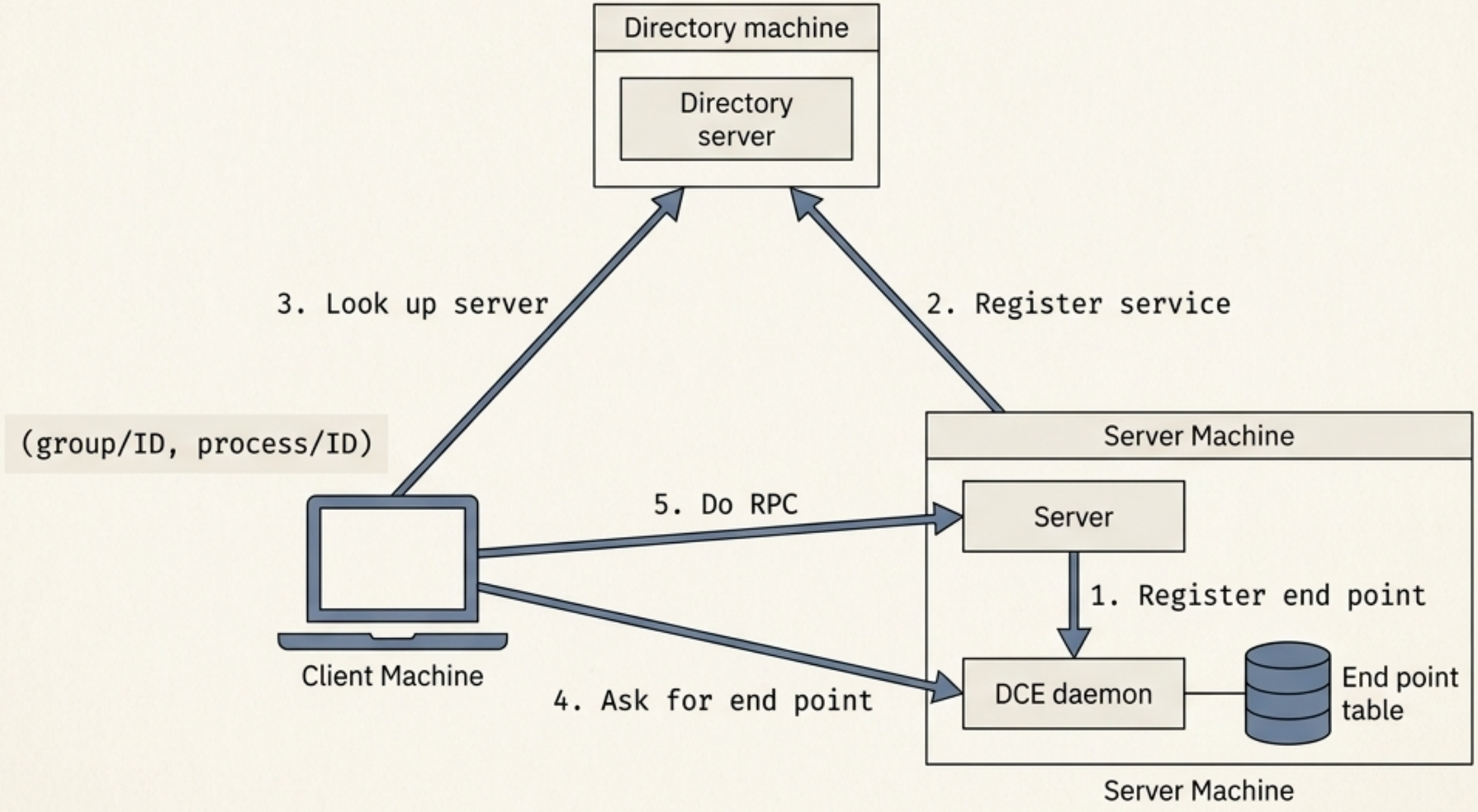
Real-World Example: Open Software Foundation (OSF) Distributed Computing Environment (DCE)

- uuidgen generates 128-bit Universally Unique Identifiers (UUIDs) based on host address and time
- DCE Daemon & End Point Table: The server registers local port endpoints with the DCE daemon; clients query the directory server to locate the host and resolve specific endpoints

Writing a client and a server in DeE RPC



DCE RPC: Locating the Server

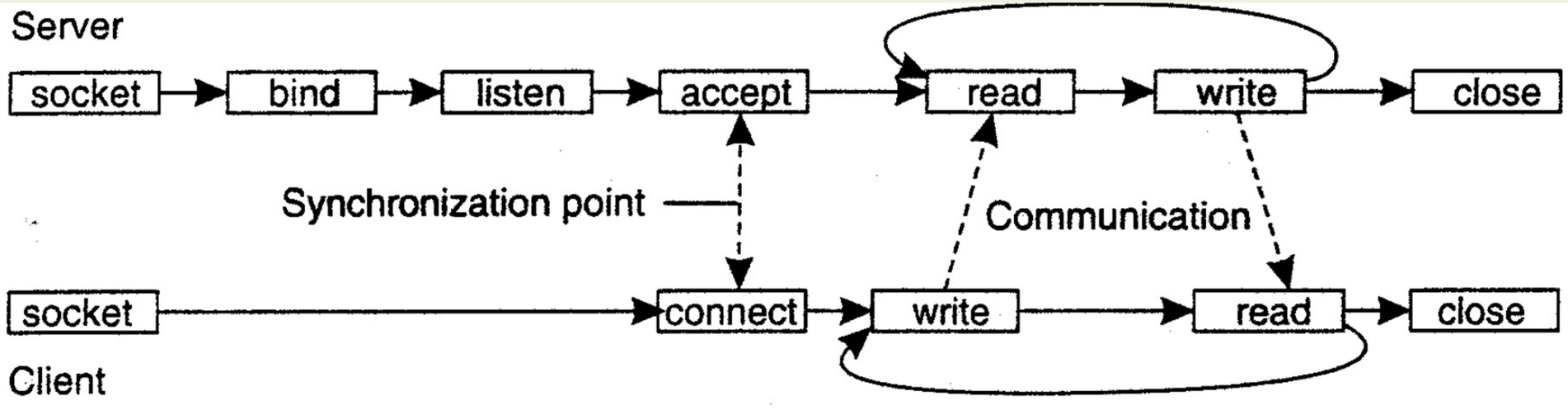


Content

- Communication Protocols
- Remote Procedure Call (RPC)
- Message-Oriented Middleware (MOM) & Stream Communication
- Naming Systems

Transient Messaging: Connection-oriented communication pattern using sockets

- **Berkeley Sockets**: Transport-level abstraction offering low-level primitives (socket, bind, listen, accept, write, read) for point-to-point connections.



Transient Messaging: Message-passing primitives of MPI

- **Message-Passing Interface (MPI)**: High-performance cluster primitive supporting advanced buffering and synchronization modes (e.g., MPI_bsend, MPI_ssend, MPI_recv) without general-purpose TCP/IP overhead

Primitive	Meaning
MPI_bsend	Append outgoing message to a local send buffer
MPI_send	Send a message and wait until copied to local or remote buffer
MPI_ssend	Send a message and wait until receipt starts
MPI_sendrecv	Send a message and wait for reply
MPI_isend	Pass reference to outgoing message, and continue
MPI_issend	Pass reference to outgoing message, and wait until receipt starts
MPI_recv	Receive a message; block if there is none
MPI_irecv	Check if there is an incoming message, but do not block

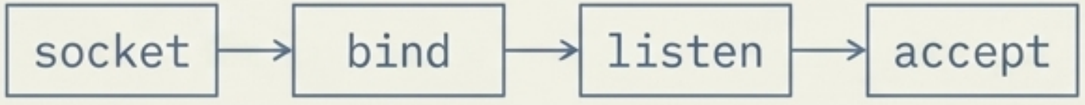
Decoupling Space & Time

Transient (Sockets/MPI)

State: Discarded if receiver is offline.

Failure Tolerance: Low (assumes stable connection).

Coupling: High (both must be active).



Persistent (MOM/WebSphere)

State: Stored in intermediate middleware queues up to minutes/hours.

Failure Tolerance: High (receiver can be down).

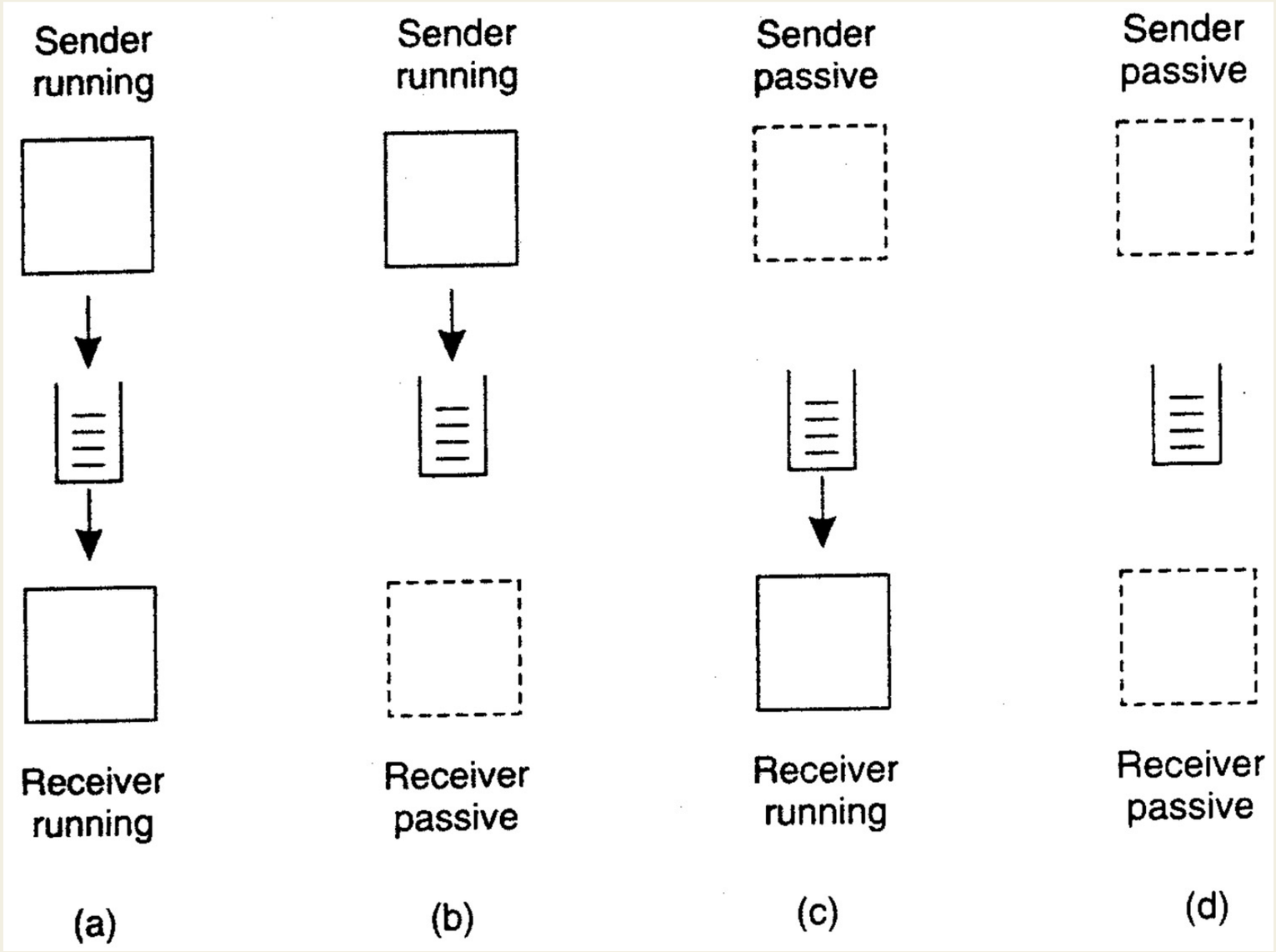
Coupling: Low (decoupled in space and time).

`MQopen, MQput, MQget, MQclose`

Message-Oriented Persistent Communication: Message Queues

- **Asynchronous Decoupling**: Senders and receivers do not need to be executing simultaneously
- **Queue Abstraction**: Senders use non-blocking put to append to a queue; receivers use get (blocking), poll (non-blocking), or notify (event-driven)

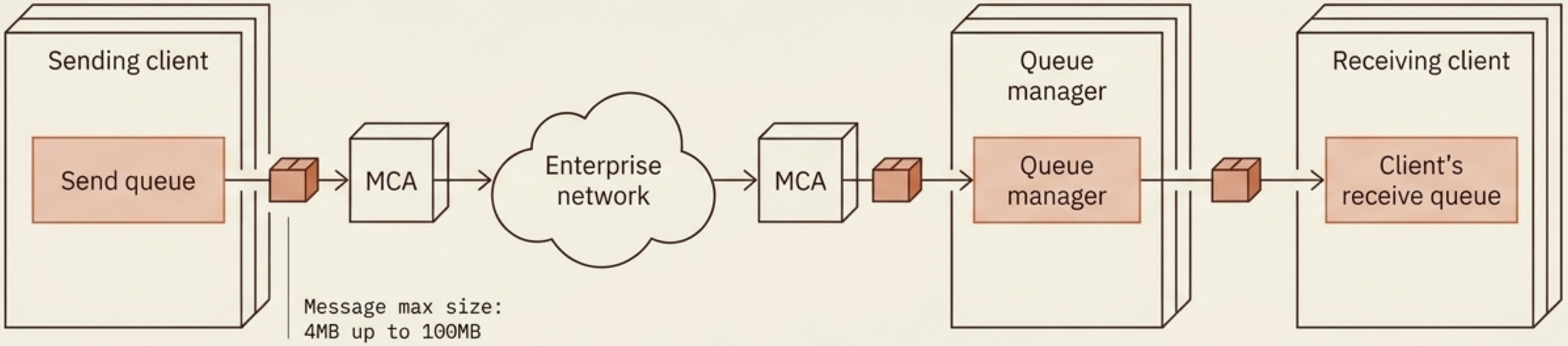
Four combinations for loosely-coupled communications using queues



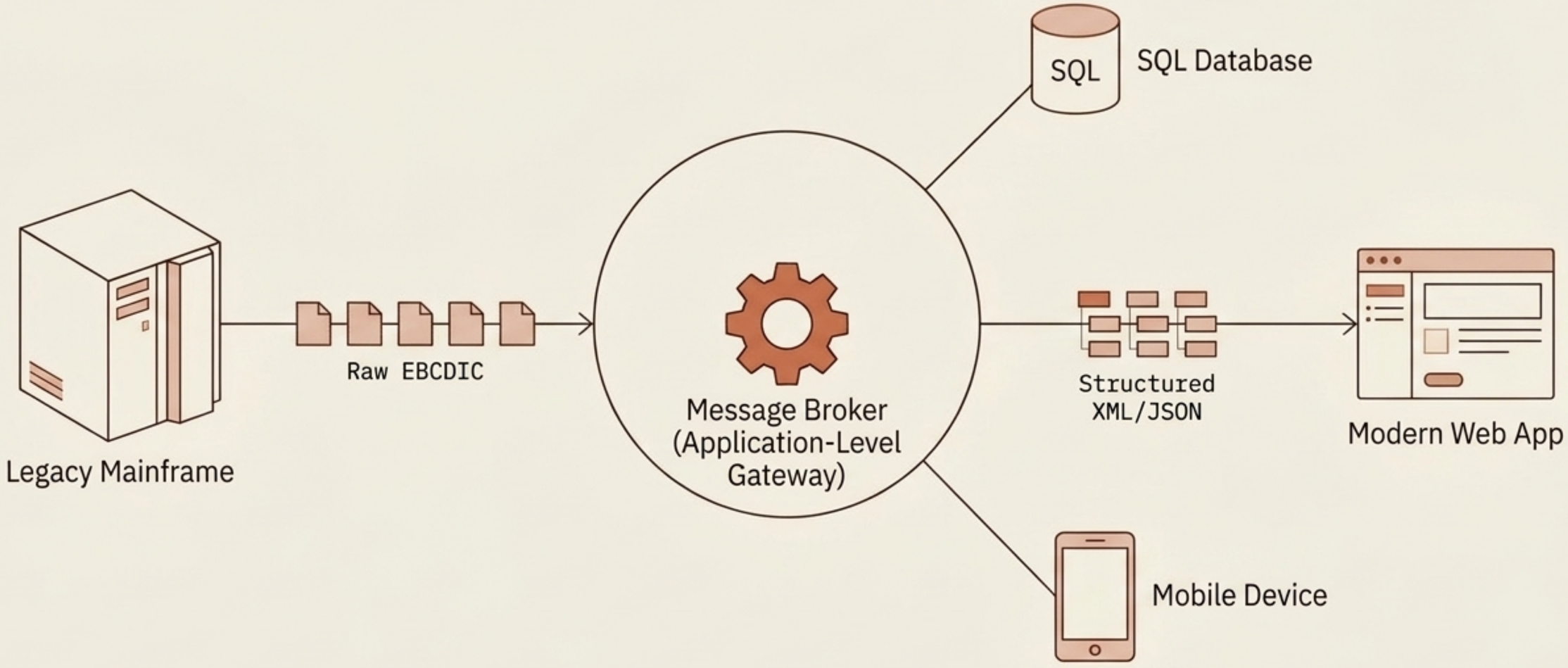
Interface to a queue in a message-queuing system

Primitive	Meaning
Put	Append a message to a specified queue
Get	Block until the specified queue is nonempty, and remove the first message
Poll	Check a specified queue for messages, and remove the first. Never block
Notify	Install a handler to be called when a message is put into the specified queue

Persistent Queueing: WebSphere MQ



The Message Broker as Translator



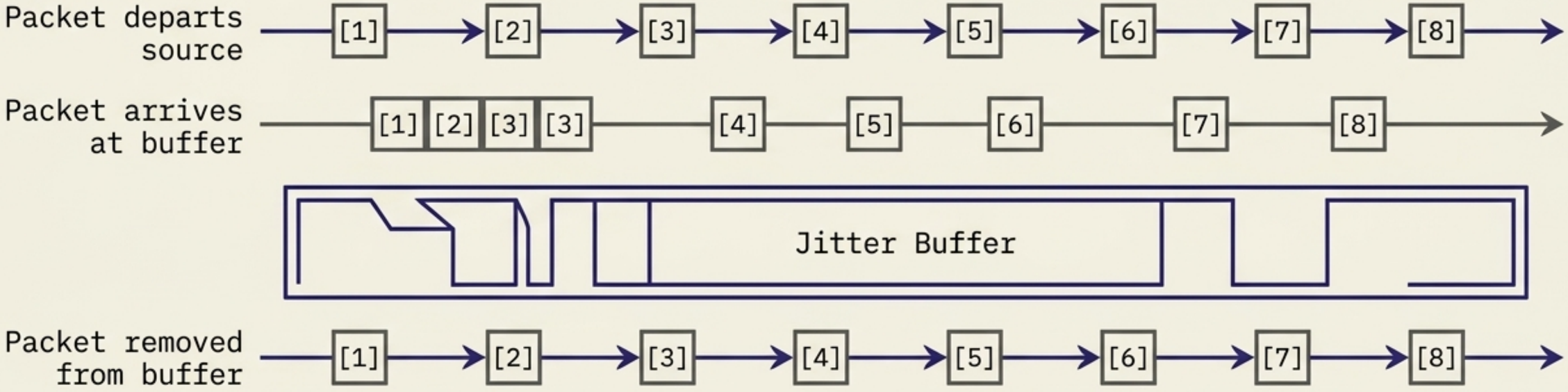
The broker acts as an application-level gateway, matching and converting formats so the queuing system remains oblivious to the data content.

When Timing is the Data

Discrete (RPC / MOM)

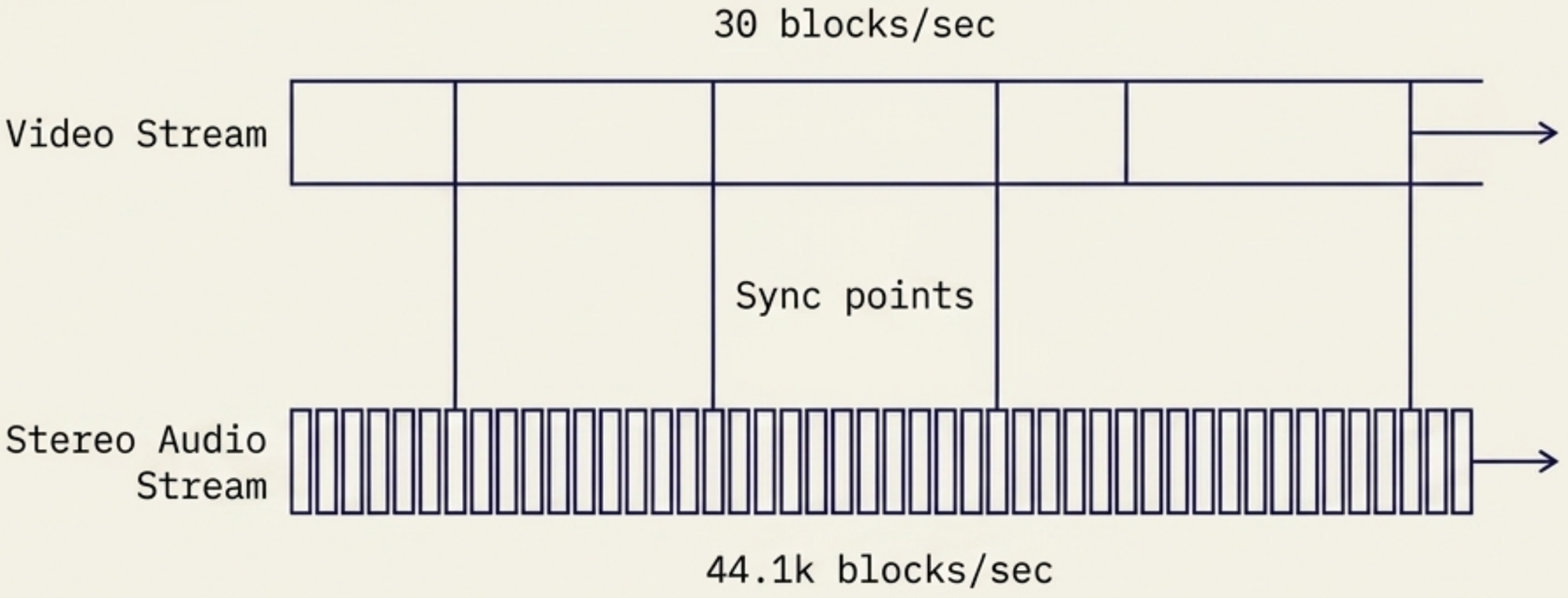
Timing does not affect correctness.

Continuous (Streams)



Gemini Notebook

Stream Synchronization



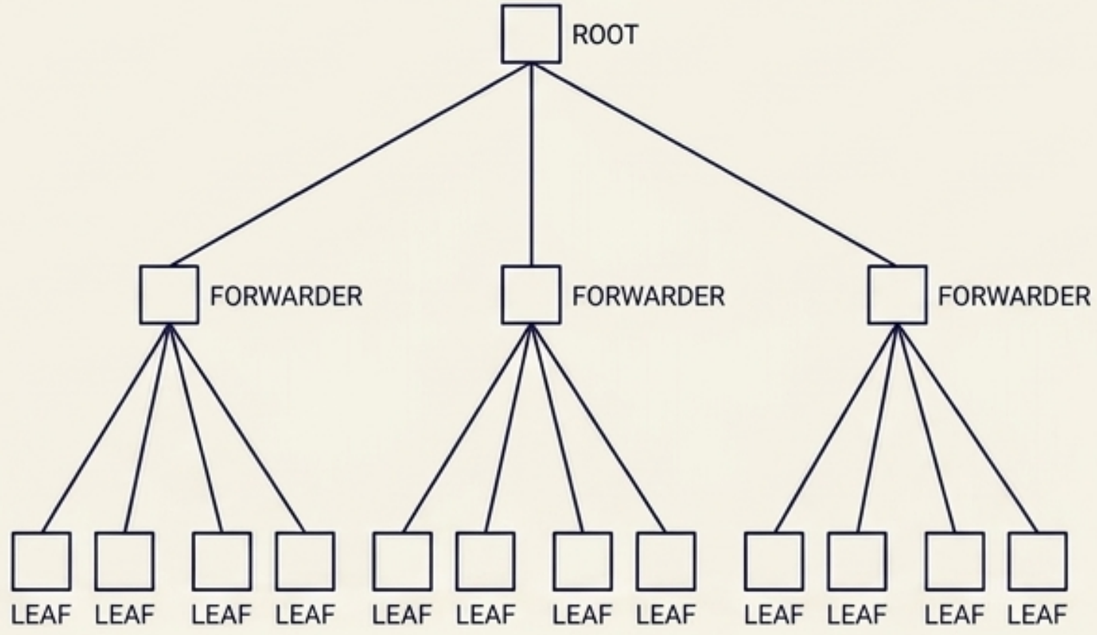
Required QoS Parameters

- Bit rate
- Delay to session
- End-to-end delay
- Jitter variance

Gemini Notebook

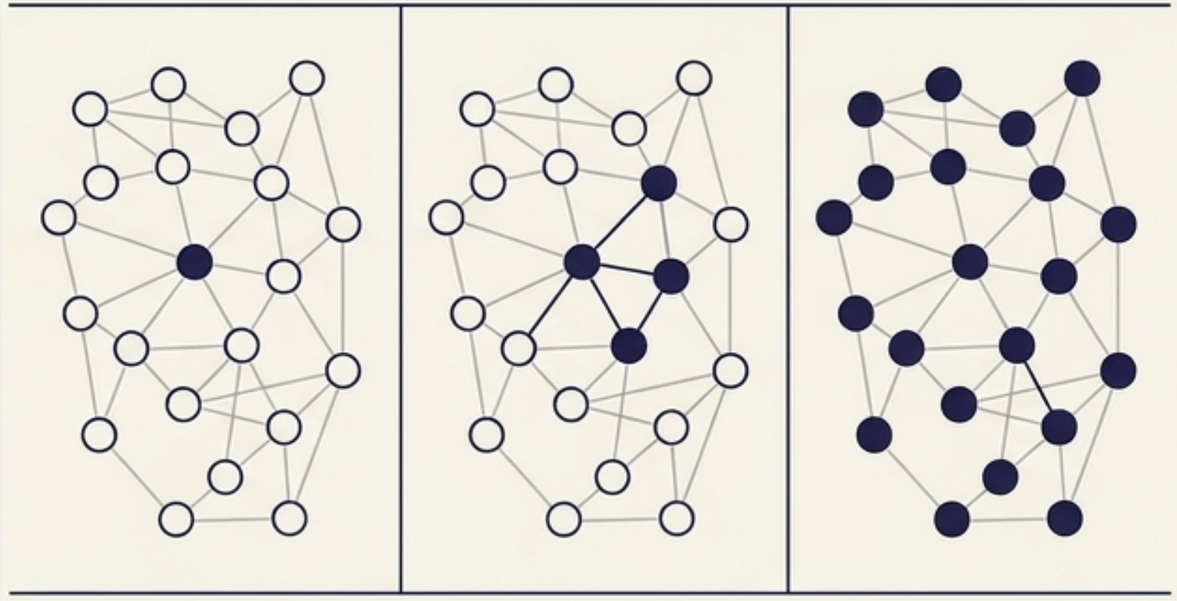
Scaling Out: Trees vs. Epidemics

Application-Level Multicasting



Structured routing, deterministic but fragile.

Gossip/Epidemic Dissemination

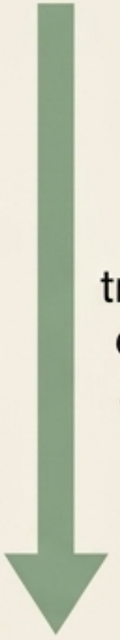


Unstructured, randomized, highly robust. Employs vector/Lamport clocks to order the chaos.

Gemini Notebook

The Taxonomy of Entity Location

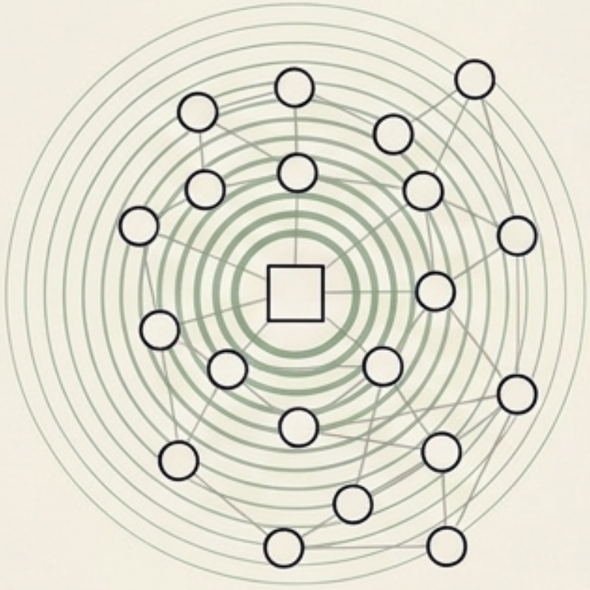
Name	Human-friendly	`printer1.cs.vu.nl`	Resolves to an ID
Identifier (ID)	Machine-friendly, unique representation. Never changes.	`0x8A4F2C99` (128-bit hash)	Resolves to an Address
Address	Location-specific. Changes if the entity moves.	`192.168.1.5:80`	Routes the payload



How do we traverse this chain efficiently across a million nodes?

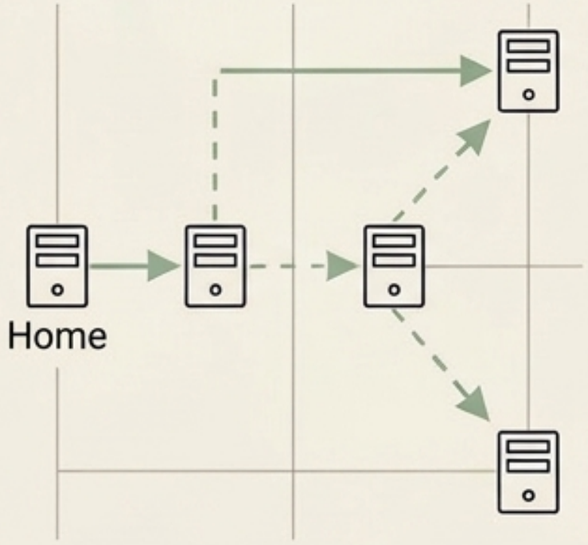
The Evolution of Flat Naming

Broadcasting → Forwarding Pointers → Home-Based



Node shouts to all:
"Who has ID X?"

Verdict: Doesn't scale beyond a LAN.



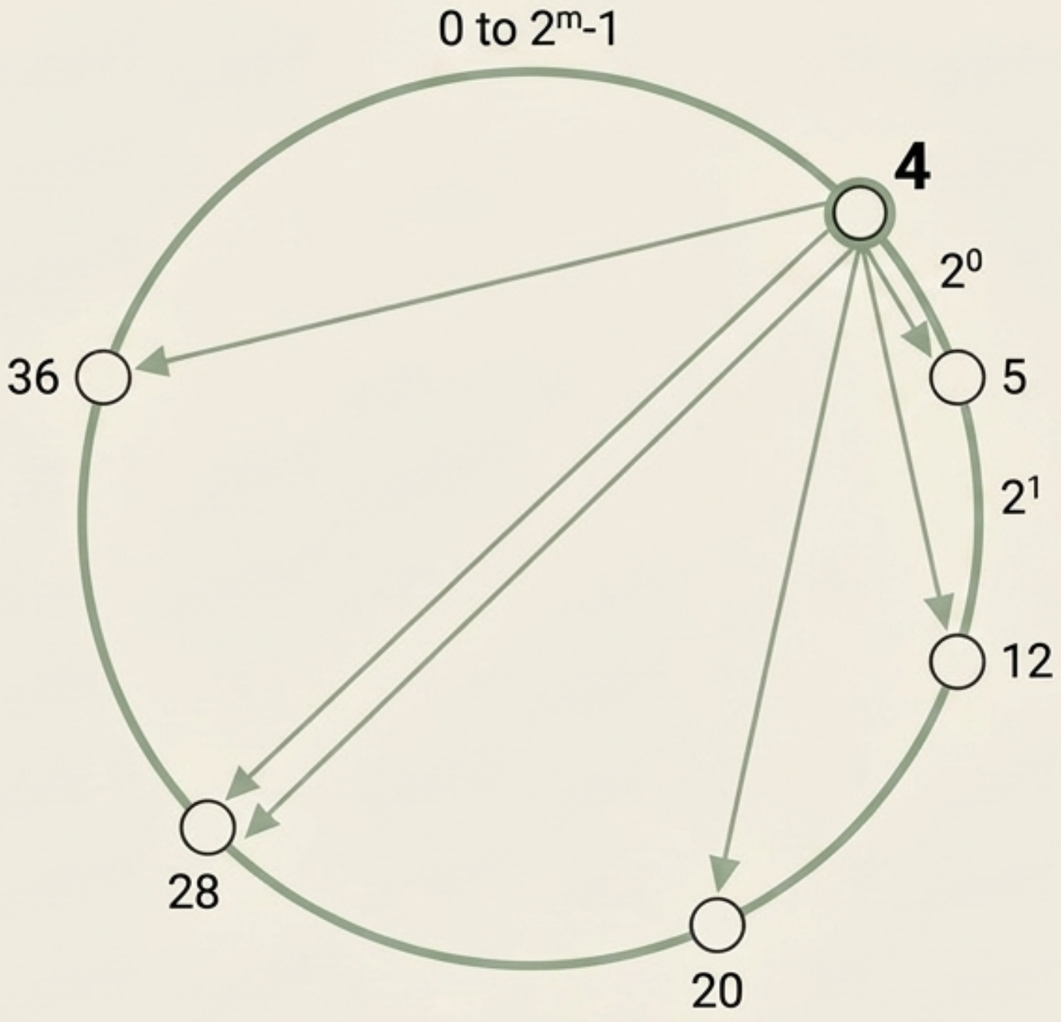
Leaves a trail of pointers behind as it moves.

Verdict: Fragile; if one link breaks, the node is lost forever.

Updates permanent Home Agent upon moving.

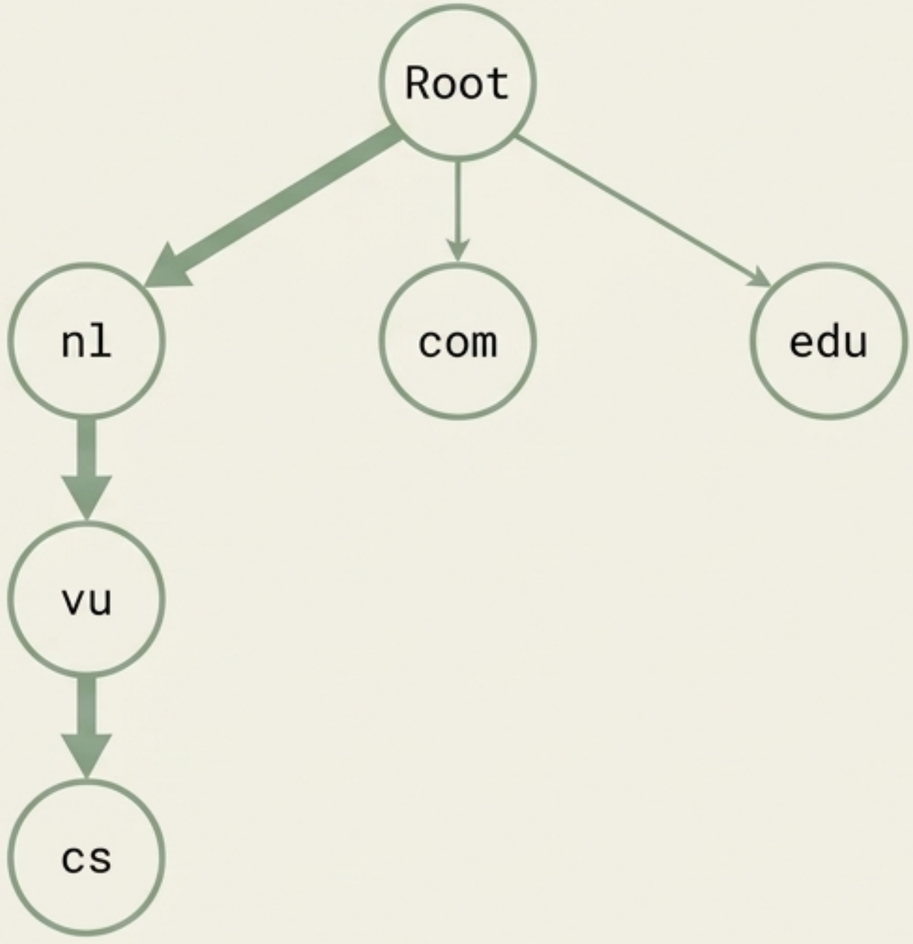
Verdict: Better, but creates a geographic bottleneck.

Distributed Hash Tables: The Chord Ring



Resolution takes O(log N) steps instead of O(N) because the finger table halves the search space every jump.

Structured Naming: Directed Graphs

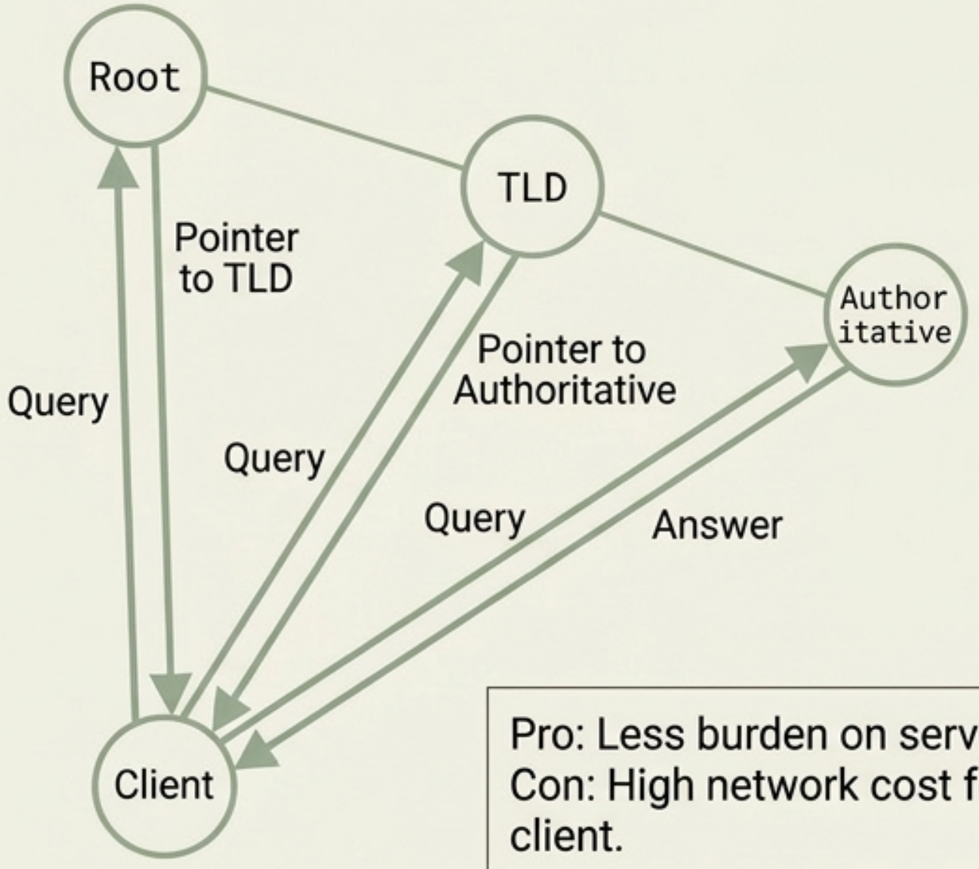


The Closure Mechanism

Knowing exactly where and how to start parsing a name. Resolution must always begin at a known, fixed context, such as the global root server.

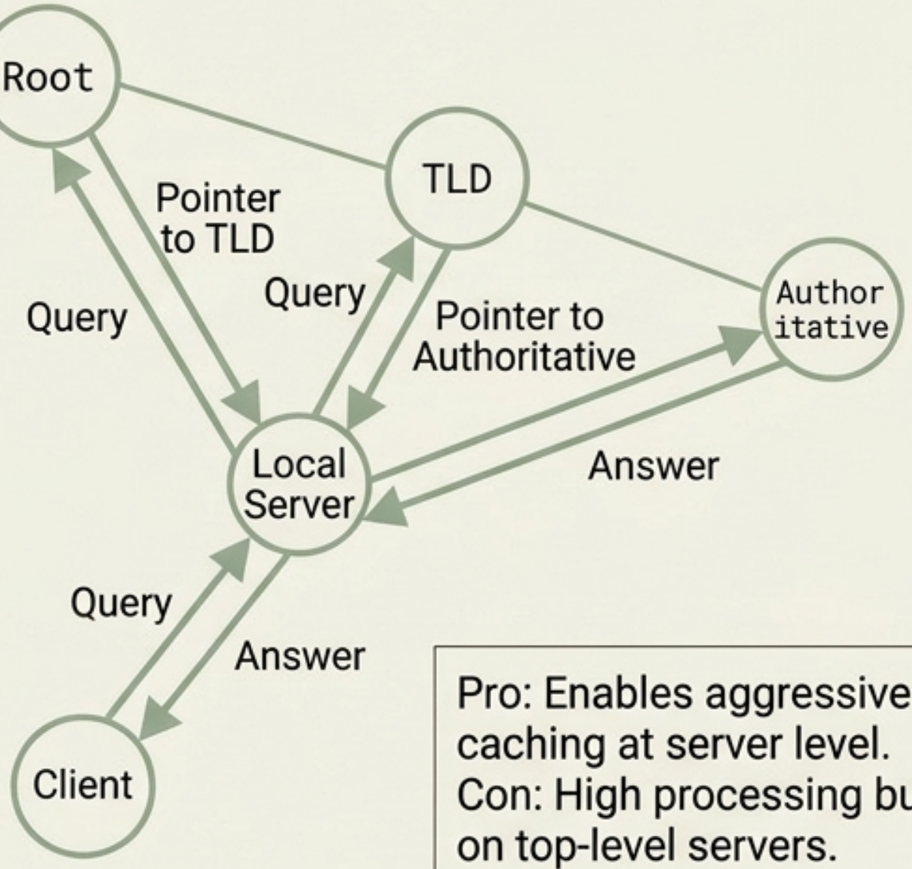
Name Resolution Strategies

Iterative Resolution



Pro: Less burden on servers.
Con: High network cost for client.

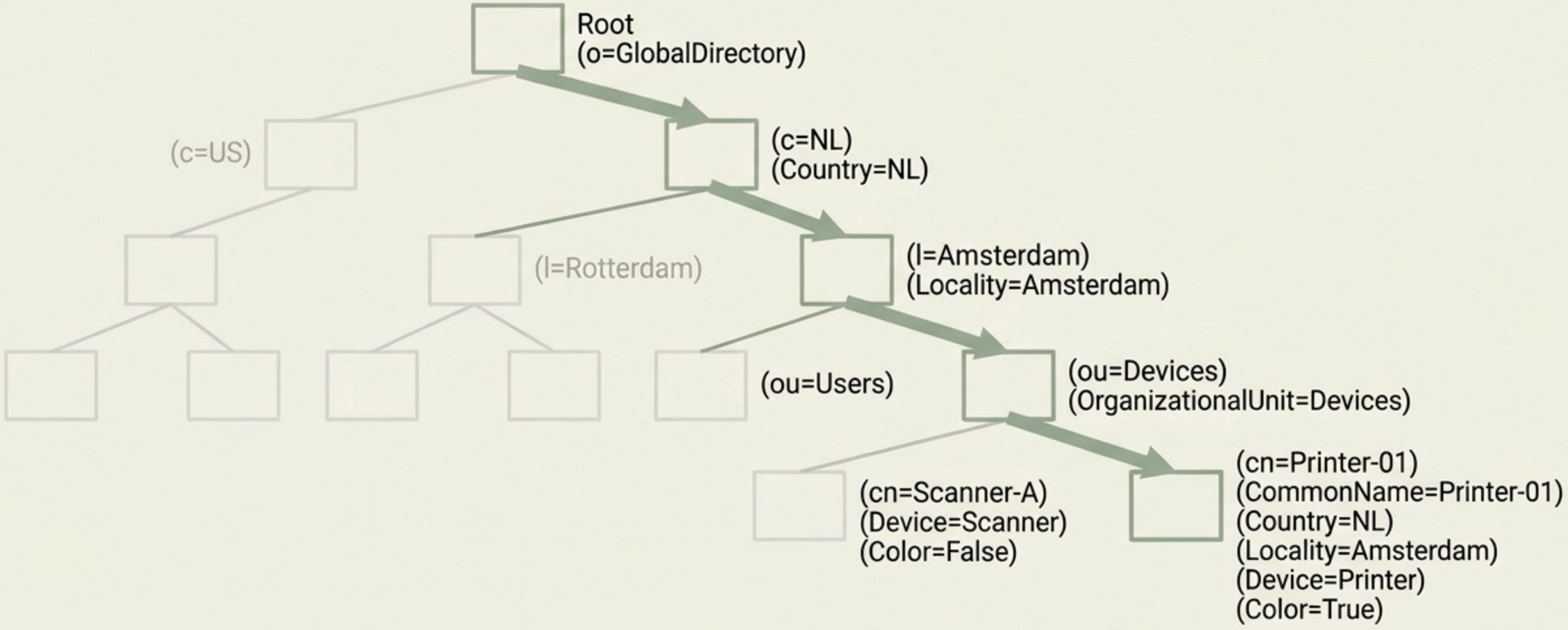
Recursive Resolution



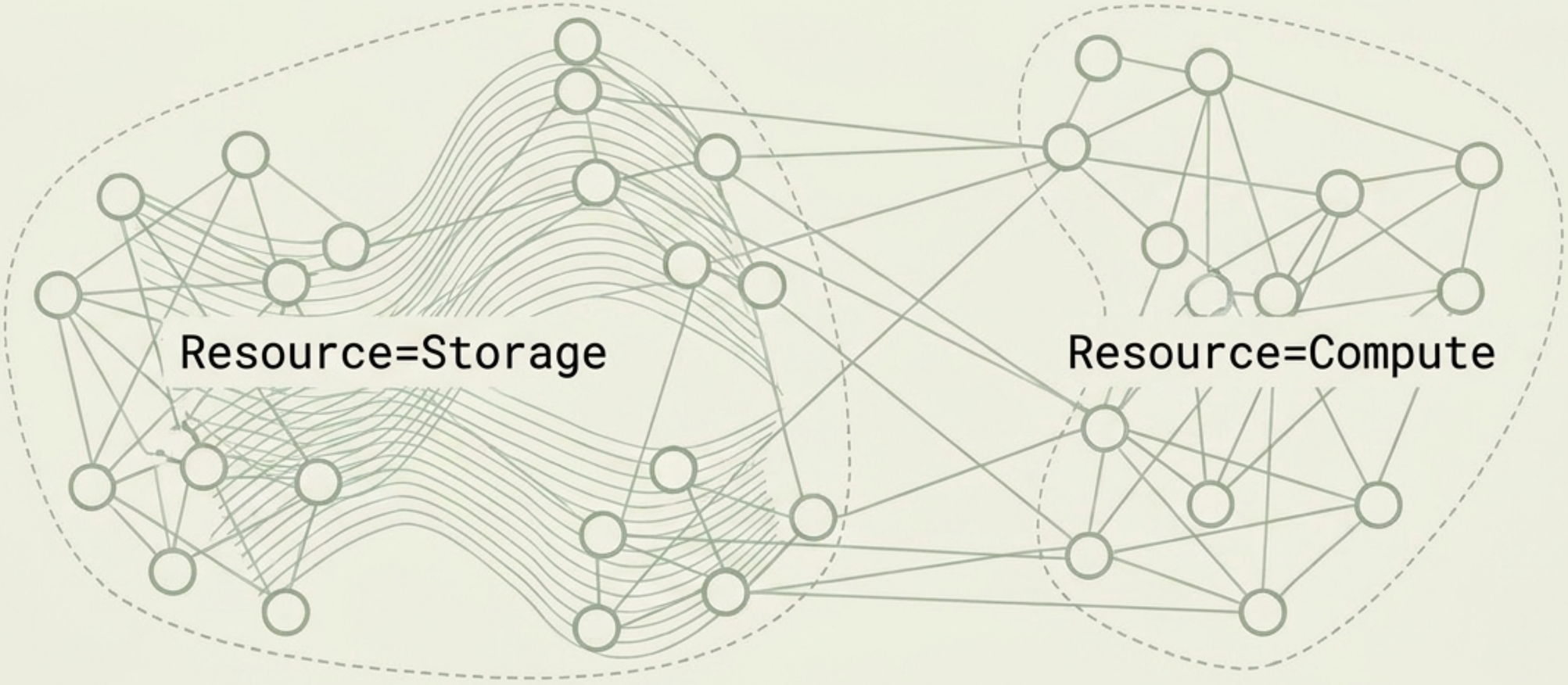
Pro: Enables aggressive caching at server level.
Con: High processing burden on top-level servers.

Attribute-Based Naming: LDAP

(&(Country=NL)(Locality=Amsterdam)(Device=Printer)(Color=True))

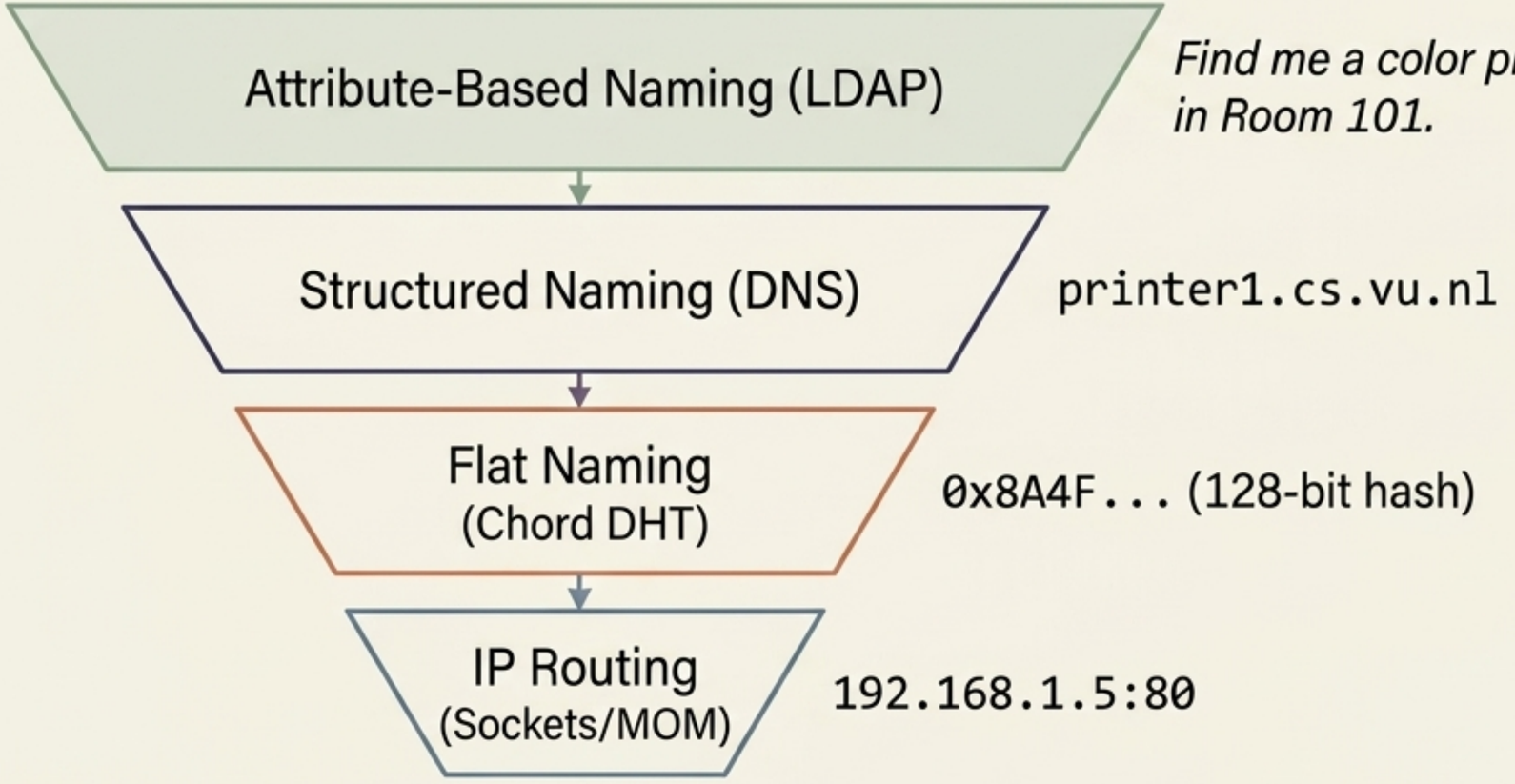


Decentralized Attribute Search



Nodes aggregate attribute queries in a decentralized swarm without a master index, using epidemic dissemination.

The Resolution Cascade



Distribution transparency is achieved through continuous, cascading resolution.

Synthesis & Practical Assignment

Review Diagnostics

1. Why must RPC client and server stubs be compiled from the identical specification?
2. Under what network conditions does MOM outperform RPC?
3. Why does Chord DHT resolution scale logarithmically rather than linearly?

Lab Briefing

Task: Implement a mini-DHT registry and messaging client.

Step 1: Use `Sockets/MPI` to establish node communication.

Step 2: Build an `O(log N)` finger table for flat name resolution.

Deliverable: A fault-tolerant system where a client can discover and message a node even if its IP address changes.